SPO reproduction: fixed endpoint still pending.

Current objective: reproduce the published RL result on our A100.

What we tested	Latest result	What it establishes
Authors' released weights	**56.69%**, paper: 56.7%	Evaluation check
Earlier own run	**22.19%** after 7,856 updates	Failed before endpoint
Fresh restart	**55.88%** at 8,336 updates	+16.10 points over SFT

Recent intermediate tests have declined

The scheduled test reached 56.85% at 8,016 updates, then fell to 56.44% and 55.88%. The latest remains above SFT but is below the paper's 56.7%. This is not the fixed endpoint or a causal test of the BOS adapter.

Next: continue to the fixed 11,040-update endpoint. Reproduction remains unfinished.

GSM8K: mean correctness over 16 answers per question; all saved answers regraded. One training run, not independent replications. Exact historical-code fidelity remains unresolved.

Before rewarding success, make success reachable.

Historical context (7 October): Teach a recursive language model (RLM) to solve problems by using Python, asking helpers, and combining their results.

The question behind our experiments

Why can reinforcement learning (RL) improve a math model, while our RLM training has struggled to produce reliable gains on new tasks?

Working explanation: Math training builds on relevant prior competence. An RLM may need help applying familiar skills in an unfamiliar workflow.

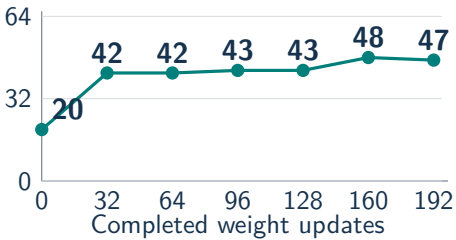
Teaching implication: Check what the learner can already do, teach the missing pieces, then give it useful practice and feedback.

RL learns from scored attempts. An RLM combines language-model calls with a code workspace. We are exploring what preparation makes this learning effective.

Math RL builds on a strong head start.

Pretraining did much of the preparation: our starting model had already learned from extensive mathematical text and worked solutions.

Correct final answers / same 64 MATH500 questions



Prior skill: it can already solve some problems.

Exploration: multiple attempts reveal successes and mistakes.

Feedback: an answer checker distinguishes them, giving training something useful to reinforce.

Historical Dr. GRPO run, 7 October: one greedy answer per question; repeatedly inspected development set. Small-scale improvement, not a full-benchmark replication. Prompt choice matters.

An RLM must combine skills in a new setting.

Knowing Python or answering questions does not guarantee knowing **our tool interface**, **what to delegate**, or **what a helper needs to know**.

Invented example: which city does Mira work in?

One document says "Mira works at Aster Lab." Another says "Aster Lab is in Oxford." To find the city, the second helper needs *Aster Lab*, not just "find the city."

An attempt can fail through a bad call, missing context, a mistaken helper, or a bad plan. A final "wrong" score does not say which one to repair.

Our history: some routines improved; reliable transfer to new combinations remained limited.

An unfamiliar workflow is a possible distribution shift, not a measured explanation of every earlier failure.

Useful practice is challenging, but within reach.

Human analogy: a child missing algebra needs more than a calculus test marked "wrong." A simpler exercise and a worked example give a place to start.

Model example: Solve $3x + 2 = 14$. An attempt answering $x = 4$ succeeds; one answering $x = 5$ fails.

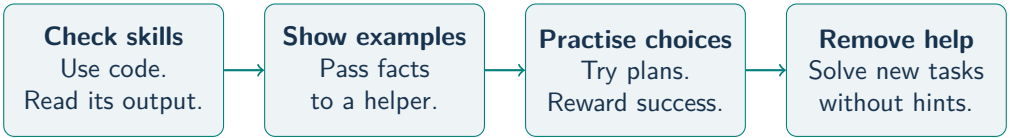
Attempts on the same problem	What the reward comparison offers
All wrong	No successful example to favor
Some right, some wrong	A contrast to learn from
All right	No distinction between attempts

Curriculum idea: choose reachable challenges and increase difficulty as skills develop.

Human learning is an analogy. The reward comparison here is specific to our method; other kinds of RL can learn from other feedback.

Teach, practise, remove help, then test transfer.

One possible approach, inspired by teaching: watch a worked example, solve with hints, then solve a new problem independently.



For the RLM: first demonstrate passing an employer's name to a helper. Later, omit that hint and ask it to decide which facts and helpers it needs.

The real test: can it use the skills in a new situation, rather than repeat a taught routine?

One hypothesis to test. Examples can be shown in a prompt or learned through supervised fine-tuning (SFT). Teach missing skills, not everything from scratch.

Different failures need different interventions.

It cannot produce a successful attempt

Clarify the interface, show a verified example, simplify the task, or try a more capable starting model. More retries alone may repeat the same failure.

It succeeds occasionally

More attempts or more varied plans may expose successes that RL can reinforce. Higher sampling temperature can add variety, but also invalid calls.

It has successes and failures, but does not improve

Check the feedback and weight updates. Does the score reward actual success? Can training identify which earlier decisions helped?

Possible remedies, not established diagnoses. More attempts cost compute; more randomness is not necessarily more useful exploration.

An open agenda of small experiments.

We are exploring, not choosing a final recipe. These are examples of questions we can test, not an exhaustive plan.

Possible explanation	Small comparison
Ability is hidden	Same weights; clearer prompt or tool example.
A skill is missing	Teach one verified helper routine; test a new case.
Teaching order matters	Same examples and budget; staged vs. shuffled.
Successes are too rare	Vary attempts or plans; keep total cost comparable.

For each: measure after preparation, then after RL. Remove hints and test new combinations. Preparation working is not yet evidence that RL helped.

Other possibilities: error-and-repair examples, stronger starting models, better feedback, and different helper roles. Change one factor; use what we learn to refine, expand or retire ideas.